SmartSense: IoT-Based Indoor Air Quality Monitoring System

A low-cost, real-time air quality monitoring prototype designed for homes and schools

Why Indoor Air Quality Matters

Indoor air can be **5-10× more polluted** than outdoor air, yet it's often invisible and ignored. Common pollutants include **dust particles**, **smoke**, **volatile organic compounds** (VOCs), **carbon monoxide**, and **excess CO_2.**

Vulnerable populations **children**, **elderly individuals**, and **students** spending hours indoors face heightened health risks from prolonged exposure to poor air quality.

The Problem We're Solving

Hidden Danger

Indoor air pollution goes unnoticed until health problems emerge

Cost Barrier

Most air quality monitors are expensive and inaccessible to average households

Awareness Gap

Limited real-time solutions for tracking and alerting about dangerous air quality

Introducing SmartSense

Our solution **monitors critical air quality parameters** in real-time and sends **instant mobile alerts** when pollution levels **exceed safe thresholds.**

Sensors

Detect dust, VOCs, CO, temp & humidity

ESP32

Processes data via Wi-Fi connectivity

Cloud

Stores readings on Firebase

Mobile App

Displays data and sends alerts

System Architecture

01

Sensor Layer

Multiple sensors continuously measure air quality parameters

02

Processing Node

ESP32 microcontroller aggregates and processes sensor data

03

Cloud Database

Firebase stores historical data and enables remote access

04

User Interface

Mobile app dashboard visualizes readings and sends push notifications

Firebase Structure

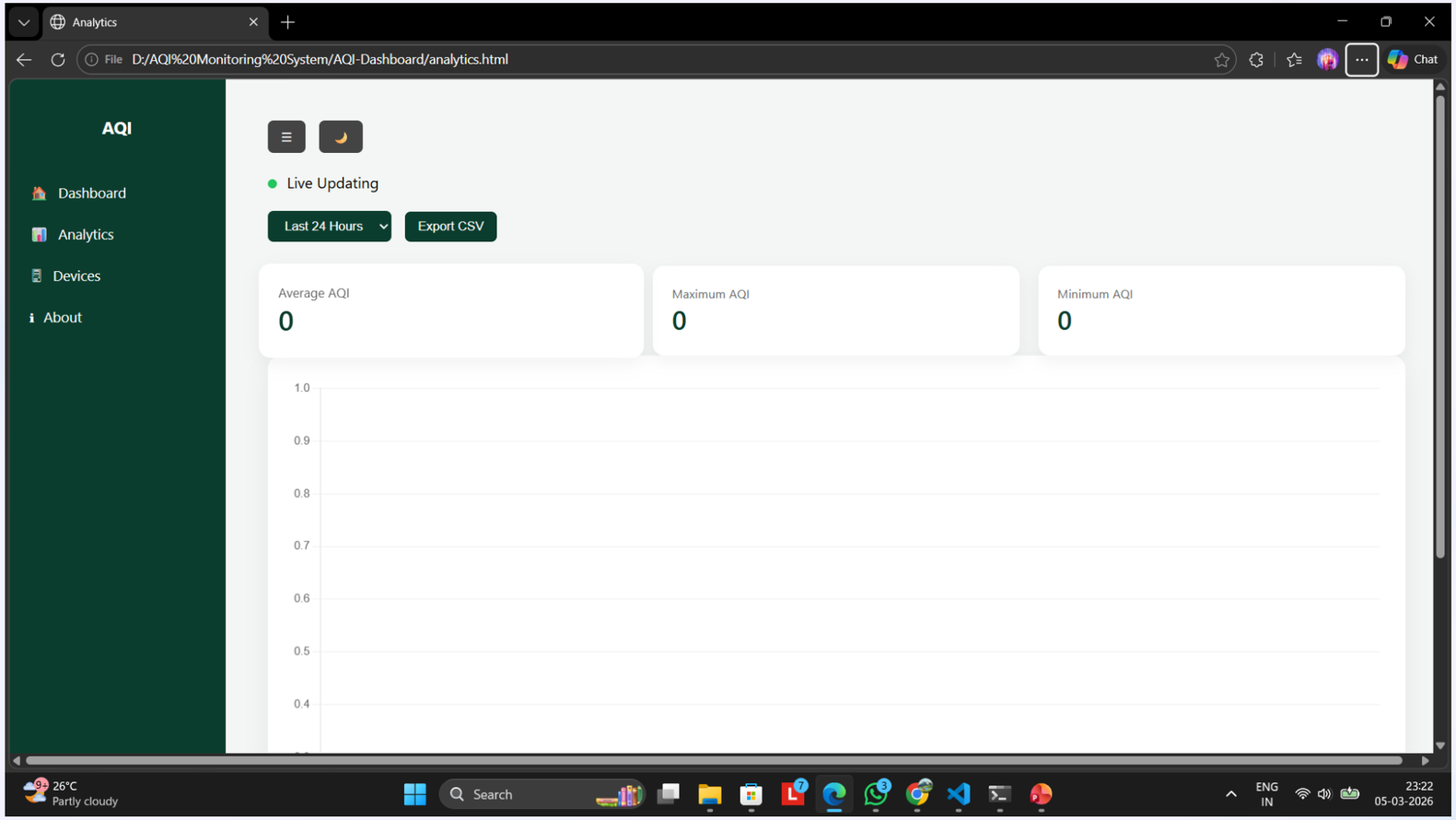
https://aqi-monitoring-sytem-default-rtdb.asia-southeast1.firebasedatabase.app

```
https://aqi-monitoring-sytem-default-rtdb.asia-southeast1.firebasedatabase.app/
▼ devices
  ▼ device01
    ▶ alerts
    ▶ bucket_10min
    ▶ bucket_1min
    ▶ live
    ▶ raw_10s
```

Hardware Components

Component	Function
ESP32	Controls sensors & Wi-Fi communication
PMS5003	Measures PM2.5 / PM10 dust particles
BME280	Monitors temperature & humidity
MQ-7	Senses carbon monoxide levels
OLED Display	Shows on-device real-time readings

Frontend Initial Progress

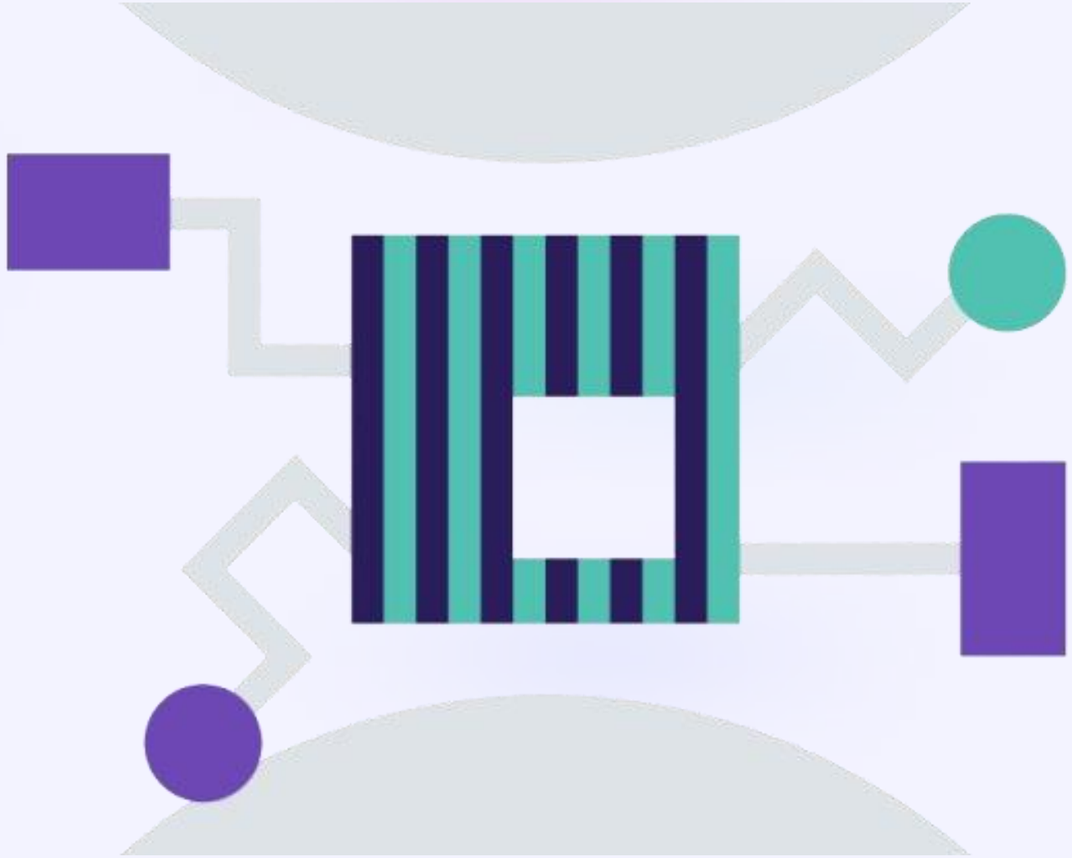


What was missing:

- Structured cloud storage
- Intelligent alert system
- Rolling time-series architecture
- Real-time frontend visualization
- Data lifecycle management

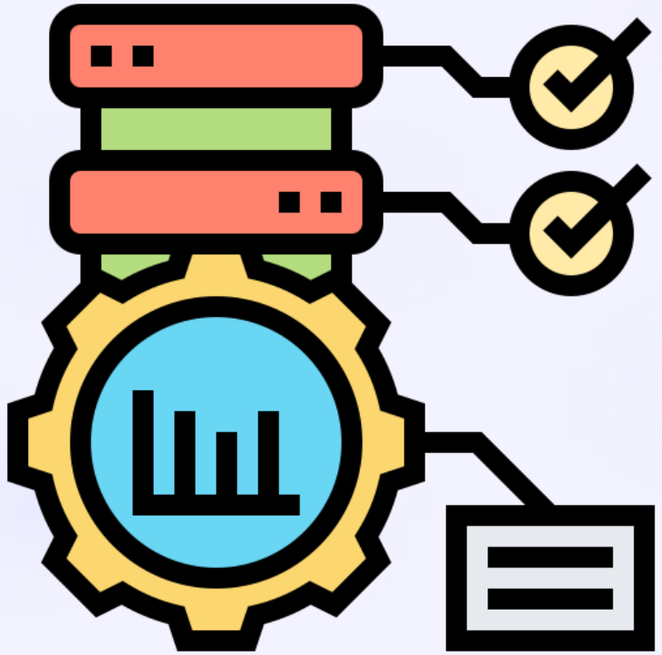
Our objective was:
To design a structured, scalable, production-style IoT monitoring architecture using affordable components.

Design Philosophy



1. Real-time but controlled data flow

Instead of sending data every second, we chose **10-second intervals**.



Why?

- Reduces database writes
- Reduces network load
- Still real-time enough for environmental monitoring

2. Rolling time-series database

We avoided infinite data accumulation.

Implemented:

- High resolution → **1 hour**
- Medium resolution → **24 hours**
- Long-term → **7 days**

This mirrors how industrial monitoring systems work.

3. Event-driven alerts instead of polling alerts

Instead of checking conditions repeatedly on frontend:

- ESP32 triggers alert once
- Stores it in /alerts
- Avoids spamming
- Resets only after recovery

This reduces unnecessary UI complexity.

4. Separation of “live state” and “history”

We created:

/live → current snapshot

/raw_10s → raw short-term data

/bucket_1min → aggregated 24hr data

/bucket_10min → long-term trend

This improves:

- Query performance
- Frontend clarity
- Logical separation of concerns

Firebase
Storage

5. Time-based structure using Unix Epoch

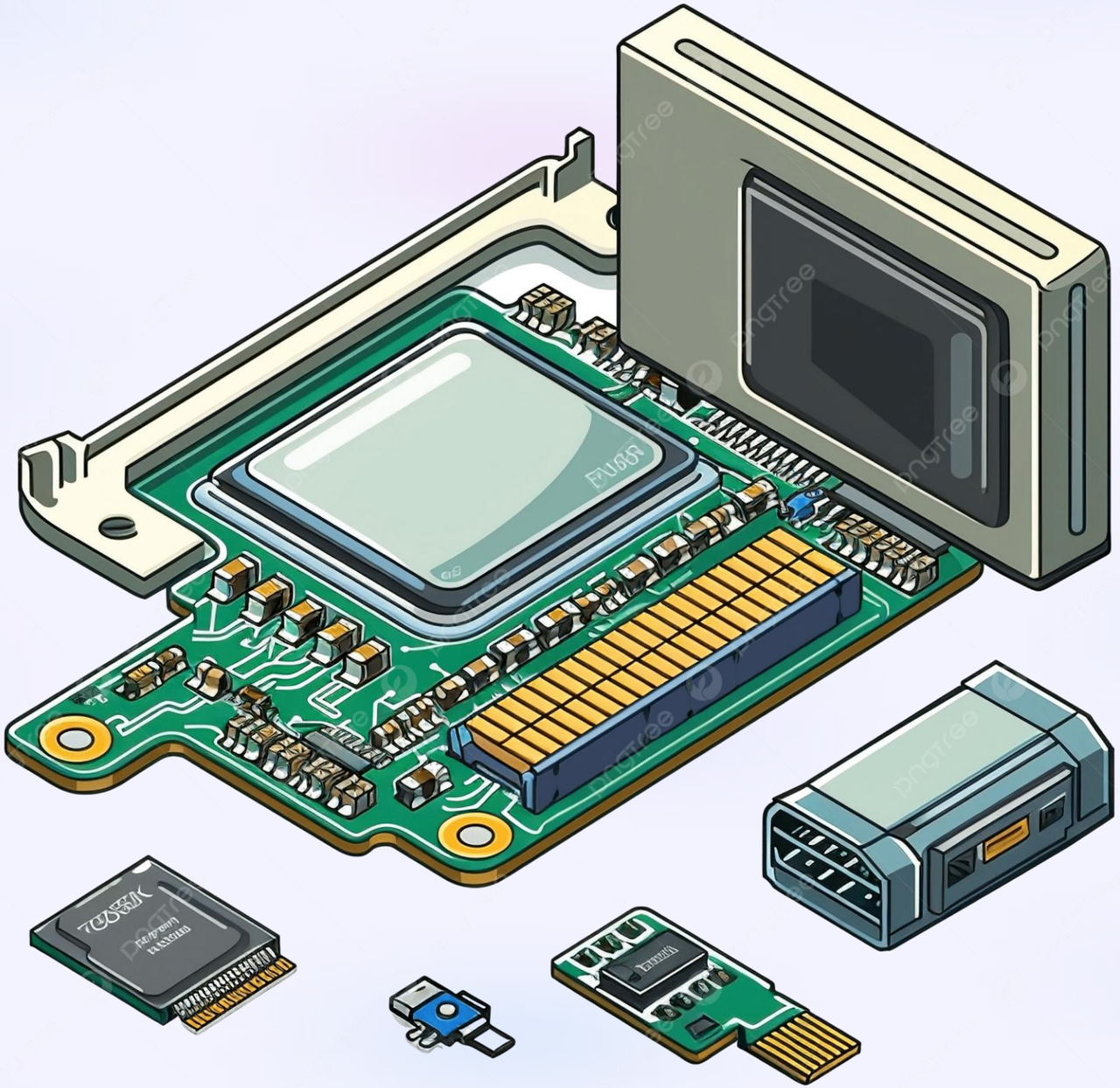
We used numeric timestamps as keys.

Why?

- Easy sorting
- Chronological integrity
- No string parsing complexity
- Works globally

Hardware Layer

Detailed Engineering



PMS5003

Challenges:

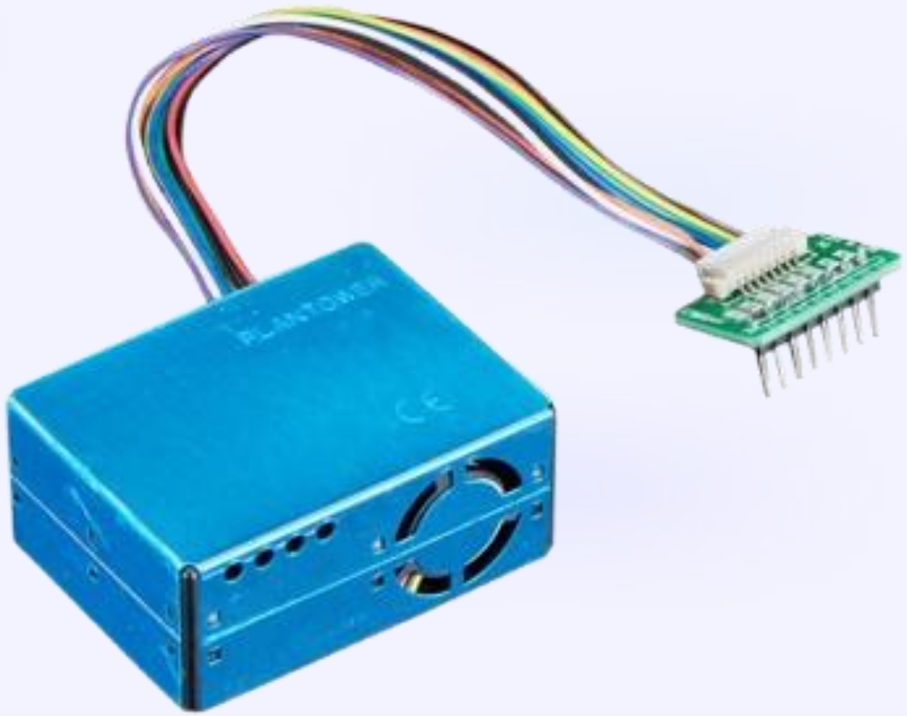
- 32-byte serial frame
- Frame alignment issues
- Incorrect header detection

Solution:

- Check 0x42 and 0x4D header
- Read fixed frame size
- Validate before parsing

Extracted:

- PM1
- PM2.5
- PM10



MQ7 CO Sensor

Challenge:

- Analog noise
- Warm-up time
- No direct ppm output

Solution:

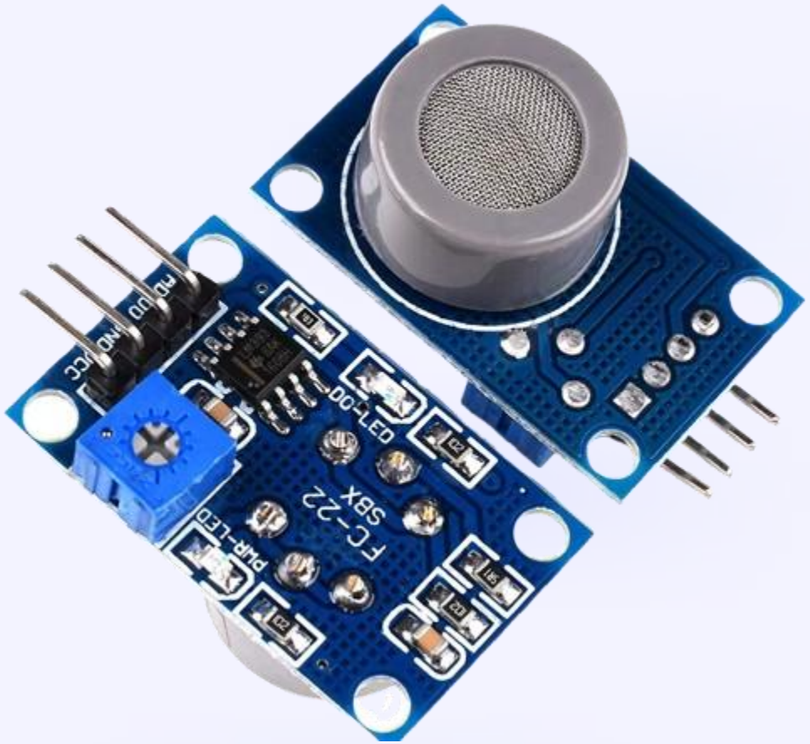
Used voltage thresholds

Defined qualitative states:

- SAFE
- ELEVATED
- HIGH

Trade off:

We prioritized relative state monitoring instead of ppm precision.



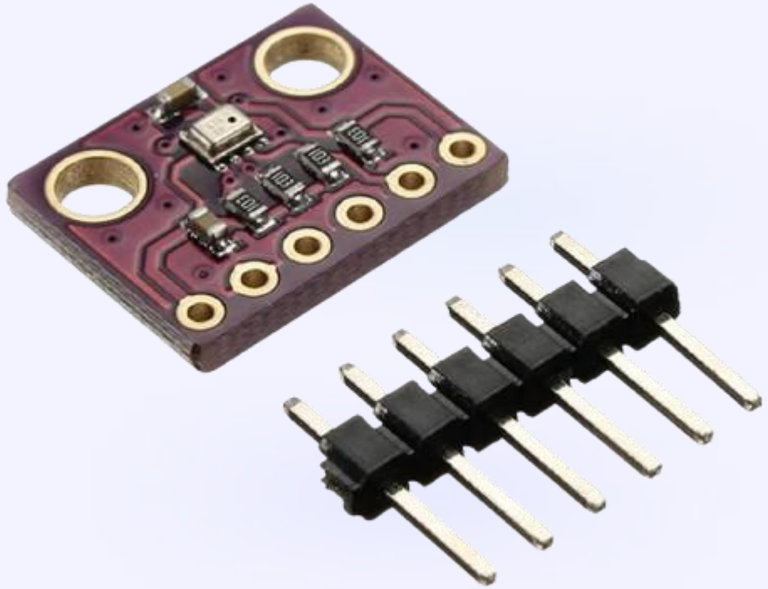
BMP280

Used for:

- Temperature
- Pressure

Added error handling:

- bmp.begin() validation
- Avoid crash if sensor missing



OLED Display

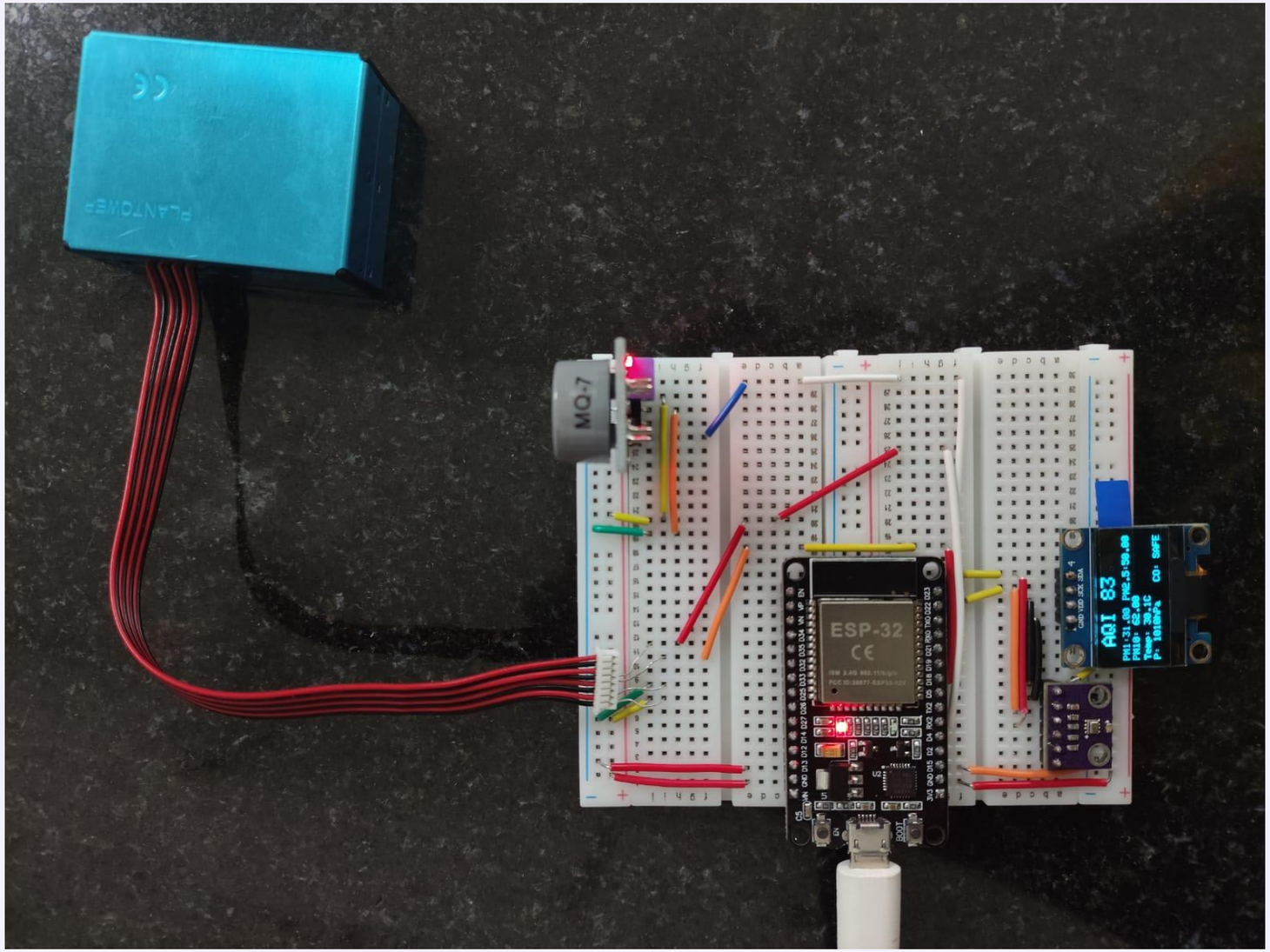
Purpose:

- Local validation
- Standalone device usability

Displayed:

- AQI
- PM2.5
- Temperature
- CO status

Hardware Implementation



AQI Calculation & Logic Equation

Our system measures the following environmental parameters:

- **PM1.0**
- **PM2.5**
- **PM10**
- **CO (Carbon Monoxide)**
- **Temperature**
- **Pressure**

The AQI value in the system is calculated using **PM2.5 and PM10 concentrations**, and the **maximum AQI value among them is taken as the final AQI**.

1. PM Sensor Measurement (PMS5003)

The **PMS5003 sensor** provides particulate matter concentration in micrograms per cubic meter.
Output values from the sensor:

$$PM1.0 = \textit{Concentration of particles} \leq 1\mu m$$
$$PM2.5 = \textit{Concentration of particles} \leq 2.5\mu m$$
$$PM10 = \textit{Concentration of particles} \leq 10\mu m$$

Units:

$$\mu g/m^3$$

These values are directly read from the sensor through **UART communication with ESP32**.

2. AQI Breakpoint Table (PM2.5)

AQI is calculated using EPA breakpoint ranges.

AQI Range	PM2.5 Range (µg/m³)	Air Quality
0 – 50	0.0 – 12.0	Good
51 – 100	12.1 – 35.4	Moderate
101 – 150	35.5 – 55.4	Unhealthy for Sensitive
151 – 200	55.5 – 150.4	Unhealthy
201 – 300	150.5 – 250.4	Very Unhealthy
301 – 500	250.5 – 500	Hazardous

3. AQI Calculation Formula

The AQI value is calculated using the **linear interpolation formula**:

$$AQI = \frac{(I_{hi} - I_{lo})}{(C_{hi} - C_{lo})} \times (C - C_{lo}) + I_{lo}$$

Variable	Meaning
C	Measured pollutant concentration
C_{lo}	Lower concentration breakpoint
C_{hi}	Higher concentration breakpoint
I_{lo}	Lower AQI index breakpoint
I_{hi}	Higher AQI index breakpoint

4. PM2.5 AQI Calculation

If PM2.5 concentration is:

$$C_{PM2.5}$$

Then AQI is:

$$AQI_{PM2.5} = \frac{(I_{hi} - I_{lo})}{(C_{hi} - C_{lo})} \times (C_{PM2.5} - C_{lo}) + I_{lo}$$

5. PM10 AQI Calculation

If PM10 concentration is:

$$C_{PM10}$$

Then AQI is:

$$AQI_{PM10} = \frac{(I_{hi} - I_{lo})}{(C_{hi} - C_{lo})} \times (C_{PM10} - C_{lo}) + I_{lo}$$

6. Final AQI Determination

After calculating both pollutant sub-indices:

$$AQI_{PM2.5}$$
$$AQI_{PM10}$$

The final AQI is determined as:

$$\mathbf{AQI_{final} = max(}\boldsymbol{AQI_{PM2.5}}\mathbf{,}\boldsymbol{AQI_{PM10}}\mathbf{)}$$

This ensures the AQI reflects the **worst air quality pollutant at that moment**.

7. AQI Classification Logic

The calculated AQI is categorized into air quality levels:

AQI Range	Air Quality
0 – 50	Good
51 – 100	Moderate
101 – 150	Unhealthy for Sensitive Groups
151 – 200	Unhealthy
201 – 300	Very Unhealthy
301 – 500	Hazardous

9. Environmental Parameter Calculation

The BMP280 sensor measures environmental conditions.

Temperature:

$$T = bmp.readTemperature()$$

Pressure:

$$P = \frac{bmp.readPressure()}{100}$$

Units:

Temperature	$°C$
Pressure	hPa

These parameters help analyse environmental conditions affecting pollution levels.

10. Dashboard Time Window Logic

Graphs display only values within the required time window.

For the last **1 hour**:

$$timestamp \geq current_time - 3600$$

For the last **24 hours**:

$$timestamp \geq current_time - 86400$$

For the last **7 days**:

$$timestamp \geq current_time - 604800$$

Various Codes Used

Imported Libraries

```
#include <WiFi.h>
#include <Firebase_ESP_Client.h>
#include "addons/TokenHelper.h"
#include "addons/RTDBHelper.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_BMP280.h>
#include <time.h>
#include <esp_task_wdt.h>
```

PM 2.5 Calculation

```
float calculateAQI_PM25(float pm25)
{
    if (pm25 <= 30)
        return (50.0 / 30.0) * pm25;

    else if (pm25 <= 60)
        return ((100 - 51.0) / (60 - 31.0)) * (pm25 - 31) + 51;

    else if (pm25 <= 90)
        return ((200 - 101.0) / (90 - 61.0)) * (pm25 - 61) + 101;

    else if (pm25 <= 120)
        return ((300 - 201.0) / (120 - 91.0)) * (pm25 - 91) + 201;

    else if (pm25 <= 250)
        return ((400 - 301.0) / (250 - 121.0)) * (pm25 - 121) + 301;

    else
        return 500;
}
```

PM 10 Calculation

```
float calculateAQI_PM10(float pm10)
{
    if (pm10 <= 50)
        return (50.0 / 50.0) * pm10;

    else if (pm10 <= 100)
        return ((100 - 51.0) / (100 - 51.0)) * (pm10 - 51) + 51;

    else if (pm10 <= 250)
        return ((200 - 101.0) / (250 - 101.0)) * (pm10 - 101) + 101;

    else if (pm10 <= 350)
        return ((300 - 201.0) / (350 - 251.0)) * (pm10 - 251) + 201;

    else if (pm10 <= 430)
        return ((400 - 301.0) / (430 - 351.0)) * (pm10 - 351) + 301;

    else
        return 500;
}
```

Pin Configurations

```
#define SDA_PIN   21
#define SCL_PIN   22
#define MQ7_PIN   19
#define PMS_RX    4
#define PMS_TX    5
```

Pin Configurations

```
#define SDA_PIN   21
#define SCL_PIN   22
#define MQ7_PIN   19
#define PMS_RX    4
#define PMS_TX    5
```

Watchdog mechanism to auto reset

```
esp_task_wdt_init(15, true);
esp_task_wdt_add(NULL);
```

Wi-fi reconnection

```
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

display.println("Connecting WiFi...");
display.display();

unsigned long wifiStart = millis();

while (WiFi.status() != WL_CONNECTED)
{
    esp_task_wdt_reset();
    delay(500);

    if (millis() - wifiStart > 20000)
    {
        display.println("WiFi Failed");
        display.display();
        delay(2000);
        ESP.restart();
    }
}
```

Real time sync

```
configTime(0, 0, "pool.ntp.org");

display.println("Syncing Time...");
display.display();

time_t now = time(nullptr);
unsigned long ntpStart = millis();

while (now < 100000 && millis() - ntpStart < 10000)
{
    esp_task_wdt_reset();
    delay(500);
    now = time(nullptr);
}

if (now > 100000)
    display.println("Time OK");
else
    display.println("NTP Timeout");
```

Auto fixing so machine restarts itself even if it was broken for some reason

```
esp_task_wdt_reset();

if (WiFi.status() != WL_CONNECTED)
    WiFi.reconnect();

time_t now = time(nullptr);

if (now < 1700000000)
    now = millis() / 1000;
```

Alert system based on co voltage thresholds

```
const char* coStatus;

if (mq_voltage < 0.08)
    coStatus = "SAFE";
else if (mq_voltage < 0.15)
    coStatus = "ELEVATED";
else
    coStatus = "HIGH";
```

Final AQI calculation

```
float aqi = max(
    calculateAQI_PM25(pm25_value),
    calculateAQI_PM10(pm10_value)
);
```

Rolling Average Method & Bucketing

Rolling Average Method Used in the Project

Air quality sensor readings often fluctuate due to:

- sensor noise
- airflow variations
- sudden environmental disturbances
- measurement uncertainty

To obtain **stable and reliable AQI readings**, the SmartSense system implements a **rolling average (moving average) method**.
This method smooths the data before it is displayed or used for long-term analytics.

Benefits of Bucketing

To efficiently store and visualize air quality data over different time ranges, the SmartSense system uses a **multi-level data bucketing approach**.

Sensor readings are collected every **10 seconds** and aggregated into larger time buckets for long-term analytics and smoother visualization.

- Reduces database storage load
- Smooths noisy sensor readings
- Enables efficient visualization across different time ranges
- Prevents excessive data transfer to the dashboard

Raw Data Bucket (10-Second Resolution)

Path: devices/device01/raw_10s

Purpose: High-resolution real-time monitoring

Sampling Rate: 1 reading every **10 seconds**

Usage: Display **last 1 hour** of detailed sensor data

Total Points in 1 Hour:

$$3600/10 = 360$$

1-Minute Bucket (Short-Term Aggregation)

Path: devices/device01/bucket_1min

Purpose: Smooth visualization and reduced storage

Aggregation: Average of **6 consecutive 10-second readings**

Usage: Display **last 24 hours**

Total Points in 24 Hours:

$$24 \times 60 = 1440$$

10-Minute Bucket (Long-Term Aggregation)

- **Path:** devices/device01/bucket_10min
- **Purpose:** Efficient long-term trend analysis
- **Aggregation:** Average of **60 consecutive 10-second readings**
- **Usage:** Display **last 7 days**
- **Total Points in 7 Days:**

$$7 \times 24 \times 6 = 1008$$

Purpose of the Rolling Method

The rolling method is used to:

- Reduce sensor noise
- Stabilize AQI readings
- Smooth graphs
- Improve trend visualization

Prevent sudden spikes from affecting the dashboard
Without rolling averaging, sensor values would appear **highly unstable and inaccurate**.

Different Phases of our Project

Phase 1: Hardware Integration and Local Monitoring

- Selected **ESP32** as the main microcontroller

- Integrated environmental sensors:

  - PMS5003 for particulate matter (PM1, PM2.5, PM10)
  - BMP280 for temperature and pressure
  - MQ7 for carbon monoxide detection

- Added **OLED display** for local real-time visualization

Phase 2 – Cloud Integration

Goal: Enable **remote monitoring and data storage**

Steps performed:

- Integrated **Firebase Realtime Database**
- Connected ESP32 to Wi-Fi network
- Implemented data upload via HTTP requests

Initial approach:

- Uploaded only **live sensor readings**

Problem discovered:

- Flat database structure
- No scalability for historical data

```
devices
└── device01
    ├── aqi
    ├── pms25
    ├── pms10
    ├── co_status
    └── temperature
```

↓

```
devices
└── device01
    ├── latest
    └── history
```

↓

```
devices
└── device01
    ├── live
    ├── raw_10s
    ├── bucket_1min
    ├── bucket_10min
    └── alerts
```

Frontend Dashboard Development

Technologies used:

- React (Vite)
- Chart.js
- TailwindCSS
- Firebase SDK

Vite + React

Chart.js

Tailwind CSS

Firebase
Storage

DevOps & Deployment

Development practices included:
- Git version control
- Branch management
- Merge conflict resolution

Security improvements:
- API key rotation after accidental exposure
- Environment variable configuration

Deployment:
- Firebase Hosting for frontend dashboard
- CI-based builds

Teamwork & Communication

- Our team maintains effective coordination through clear communication and task division.

- We **conducted weekly meetings to discuss progress, share updates**, and **reassign tasks if needed**.

- All files and documents are maintained on a **shared Google Drive folder** **(Drive link)** for transparency.

- **Each member contributes both technically and creatively** ensuring the project remains balanced between electronics, coding, and design.

- GITHUB **Repository Link**

Work Division

Task	Responsibility
Assembling Hardware	Aaron, Kishor
Hardware Code(Arduino IDE)	Kishor,Pratheerth,Akshat
Frontend development	Saubhagya, Akshat
UI development	Saubhagya
Backend development	Kishor, Saubhagya
Database Creation	Kishor, Saubhagya, Pratheerth
Device testing	Kishor, Aaron
PPT Creation	Saubhagya, Kishor

Conclusion

SmartSense demonstrates a **practical, low-cost IoT approach** to indoor air quality awareness. Our prototype proves that **real-time monitoring** and alerting **can be accessible to homes and schools**, **promoting healthier indoor environments** for everyone.